

Homework 1 - Deterministic Search

Introduction

The university of Roboland wants to use a robot to transport students' tests to the scanning system. However, the robot has limited battery life, which often requires charging in order to get to the destination. In addition, uneven floor and robovacs in the university make the robot's task harder. To achieve the goal most efficiently, you must make use of the search algorithms shown in class, with the first task being modeling the problem precisely.

Environment

The environment is represented as a rectangular grid, with details provided in the "Input" section as a dictionary. Each cell within this grid symbolizes a different region in the world and can have different properties:

1. **Passable (P):** A passable region – the robot and other entities (robovacs, charging stations, uneven floor tiles) can be there.
2. **Impassable (I):** An impassable region - the robots and blockades cannot be there.
3. **Destination (D):** The location of the Destination.

The core objective is to navigate strategically to reach the destination with the minimal number of turns.

Each action in this environment occurs in discrete turns, altering the state of the environment and depicting the current positions of the robots and robovacs.

Actions

You assume control of the robot, which can perform various actions to navigate the world and charge:

1. **Move:** The robot can move up to one tile vertically (up or down) or horizontally (left or right) on the grid. It cannot move diagonally, and cannot move into an impassable tile. The action's syntax is ("move", (x, y)). For example, if you want to move the robot to a **tile** (0,2), the correct syntax for this action is ("move", (0, 2)).
2. **Wait:** A robot can choose to wait, which does not change anything about its state or position. The syntax for this action is ("wait").
3. **finish:** when the robot reaches the destination it can finish the game. The syntax for this action is ("finish").
4. **Charge:** when a robot is at a charging station, it can choose to charge to regain a certain amount of energy, charging will lead to the charging station to be inactive for a given number of turns ("תורות"). The correct syntax for this action is ("charge").

Each of the above actions is performed in turns or timestamps, allowing the environment and other entities like blockades to react or change positions based on the chosen action. Performing an Illegal action is invalid and would result in a non-valid solution, not earning score.

Additional Rules

1. **Battery Life:** the robot starts its journey with a certain number of steps remaining, representing its

battery life. If the number of steps remaining reaches 0, the robot will no longer be able to move, ending the game if not in the destination or in a charging station. Each move action would cost battery and would reduce the amount of steps remaining by one (except for uneven floor tiles as explained in point 5). The battery also has a maximum capacity, which might not equal the initial number of steps stored in it. Battery life does not affect and is not affected by actions who are not "move".

2. **Charging:** In some locations in the environment there are charging stations. If the robot is in a tile with a charging station it can choose to charge. After charging, charging stations would be refueling, resulting in the station being inactive for a certain number of turns, meaning that charging in it would not be possible at that time. It is not possible to charge the battery above its capacity, though charging would still be a legal action.
3. **Encounter with Robovacs:** if the robot is at the end of its turn in the same location as a robovac, it would hit it, damaging the system and causing it to lose battery, resulting in a loss of steps. For example, if a robot loses 5 possible steps when it encounters a robovac, and at the start of the turn it has 9 steps remaining, after moving into the location (costing 1 steps) and encountering a robovac, it will have 3 steps remaining for the next turns. If that space had 2 robovacs, it would have lost 11 possible steps, meaning the game's end. It would have still counted as the robot having performed 1 turn. If the robovac encounter requires more battery than in the robot, this will result in the robot's destruction and end the game.
4. **Robovac Movement:** robovacs move only on passable regions. They move along a pre-determined path, which is provided in the input. They can wait or move one tile vertically or horizontally (no diagonal moves) per turn in the passable tiles. If a robovac has a path of $A \rightarrow B \rightarrow C \rightarrow D$, instead of returning to point A after reaching point D, it will move to point C, and continue retracing the path in reverse, i.e., $D \rightarrow C \rightarrow B \rightarrow A$, and then again $A \rightarrow B \rightarrow C \rightarrow D$, and so forth. So, the path will be $A \rightarrow B \rightarrow C \rightarrow D \rightarrow C \rightarrow B \rightarrow A \rightarrow B \rightarrow \dots$
5. **Uneven Floor Tiles:** if the robot moves into or out from a location with uneven floor tiles, entering or exiting the space would take more battery. If a robot takes 2 steps to move into uneven floor tiles (amount would be given in input), performing the moving action would cost 2 steps instead of 1, and it would count as performing 1 turn to the turns total. The penalty is only for entering or exiting the space, waiting or charging at uneven floor tiles would not do that. Moving from one uneven floor tile to another does not carry extra penalty (of exiting or entering).
6. **Steps vs Turns:** steps are the number of battery units used by the robot, including uneven floor penalty, those are important because of the robot's battery, only allowing a certain amount of moves before dying. Turns ("תורות") are the number of actions performed by the robot (including non-moving actions), you are expected to minimize the number of **turns**.

Further Clarifications

1. Some robot problems may be unsolvable due to battery issues. You are expected to return the output for unsolvable problems for this case, as detailed in the "output" section.
2. The destination will always be reachable from the starting location of the robot (in the sense that the way is not blocked by impassable tiles, it still might not be solvable).
3. Robovacs can walk into charging stations. If the robot ends its turn in a charging station and there are robovacs that ended their turn there as well, they will drain the battery of the robot as per normal encounter. Robovacs battery drain occurs after the action in calculation such that for example if the robot has 2 battery and it gains 4 from charging and loses 3 from a robovac encounter, it will end with 3 battery (steps remaining) this will not end in a loss.
4. Robovacs paths will never include the destination. Destinations will never be uneven floor tiles.
5. The battery cannot go below 0 (so if for example the robot had 3 battery moves left, it cannot perform an action that takes 4 or more battery moves, including actions that would cost too

much battery life due to robovac).

6. Order of events:

Choosing an action	Action results (moving, Charging, waiting, uneven floor tiles)	Robovac movement	Robovac damage	Choosing an action
--------------------	--	------------------	----------------	--------------------

Goal

The overall goal is to get to the destination while minimizing the number of **turns**.

Input and the task

As an input, you will get a dictionary that describes the initial environment.

“map”: A list of lists of strings, where ‘P’ represents a passable tile, ‘I’ represents impassable tiles, and ‘D’ represents the destination tile.

“robot”: Contains the initial position of your robot (starting_location), the initial battery life (starting_moves_left), the maximum battery life (maximum_moves_left_possible), the number of possible moves lost due to a robovac (robovac_battery_damage), the number of moves taken due to uneven floor (uneven_floor_penalty).

“robovacs”: Contains the paths for every robovac, representing their pre-determined movement pattern through the grid. Every robovac starts in the first location in its path.

“charging_stations”: Contains the locations of the charging stations, as well as how much they charge gain each use and how much time they take to refuel.

“uneven_floor”: Contains the locations of all uneven floors.

Full example of a dictionary:

```
{
  "map": [
    ['P', 'P', 'I', 'P', 'P'],
    ['I', 'P', 'P', 'P', 'P'],
    ['P', 'P', 'I', 'P', 'P'],
    ['P', 'P', 'P', 'P', 'P'],
    ['P', 'P', 'P', 'P', 'D']
  ],
  "robot": {'starting_location': (1, 1), 'starting_moves_left': 9, 'robovac_battery_damage': 5,
            'uneven_floor_penalty': 3, 'maximum_moves_left_possible': 20},
  "robovacs": {'robovac1': [(0, 1), (1, 1), (1, 2)], "robovac2": [(1, 4), (2, 4), (3, 4)]},
  "charging_stations": [{"location": (0, 4), 'charge_amount': 4, 'charge_wait': 6},
                        {"location": (4, 1), 'charge_amount': 3, 'charge_wait': 5}],
  "uneven_floor": [(4, 1), (4, 2)]
}
```

This input is provided to the constructor of the class **RobotNavigationProblem** as the variable “initial”. This variable is then used to create the root node of the search, hence you will need to transform it into your representation of the state before the **search.Problem.__init(self, initial)** line.

Implementation Requirements

You will need to implement the following functions in the **RobotNavigationProblem** class:

1. **def actions(self, state):**
 - Returns all available actions from a given state.
2. **def result(self, state, action):**
 - Returns the next state, given a previous state and an action.
3. **def goal_test(self, state):**
 - Returns 'True' if a given state is a goal, 'False' otherwise.
4. **def h(self, node):**
 - Returns a heuristic estimate of a given node.

In addition you will need to implement this function:

5. **def astar_search(problem, heuristic):**
 - Returns the goal node if found, None otherwise.

Note on State Representation

You are free to choose your own representation of the state. However, it must be Hashable. We strongly suggest sticking to the built-in data types and not creating a new class for storing the state. Nevertheless, you might find unhashable data structures useful, so consider functions that transform your data structure to a hashable one and vice versa.

Evaluating your solution

Having implemented all the functions above, you may launch the A* search by running check.py. Your code is expected to finish the task in 60 seconds for every problem.

Output

When running your check.py file, you may encounter two types of outputs:

- (-2, -2, None) – No solution was found.
- A solution of the form (number of turns, run time, list of actions taken).

Code handout

Code that you receive has 4 files, you should only modify ex1.py as this is the only file you submit:

1. ex1.py - the main file that you should modify, implements the specific problem.
2. check.py - the file that includes some wrappers and inputs, the file that you should run.
3. search.py - a file that has the implementation of some basic search algorithms, including the problem and node classes.
4. utils.py - the file that contains some utility functions. You may use the contents of this file as you see fit.

Note: we do not provide any means to check whether the solution your code provided is correct, so it is your responsibility to validate your solutions.

Note: You may use any package that appears in the [standard library](#). However, the exercise is built in a way that most packages will be useless. Any other packages are not allowed. Use python 3.10 version for running the code.

Machines

In the first week of this exercise, you will get machines, you can use them to run the program and check your running times in the same way we will check them. Guidance on the machines will be given when you get the machines.

Submission and grading

You are to submit only the file named ex1.py as python files (no zip, rar, etc.). We will run check.py with our own inputs, and your ex1.py.

The check is fully automated, so it is important to be careful with the names of functions and classes. The grades will be assigned as follows:

- 85 points – Solving all the non-competitive problems under 60 seconds with the optimal solution. Suboptimal solutions will get partial grade.
- 25 points – competitive part. Computing the optimal plans (minimal number of turns) in under 60 seconds, for as many problems as possible. The grade will be relative to the solutions of other students, as well as to benchmark solutions by the TAs. The problems which will be tested might be larger than in the non-competitive part.
- Notice, you can get more than 100 points in this HW. Scores above 100 will **not** be rounded down to 100.
- The submission is due to 10.12.2025.
- Submission in pairs only or alone (alone only with special approval from Reshef or Doron). Submission in larger groups is not approved. You would have to submit with different pairs for the next exercises.
- Write your ID numbers in the appropriate field ('ids' in ex1.py) as strings. If you submit alone, leave only one string.
- When grading, we check the validity of your solutions.